

Agent Security Policy (ASP): The Missing Trust Boundary Layer in Agentic AI Systems

Kameswara Prasad Mukkamala (Prasad MK)

Independent Researcher, Rocklin, CA, USA

ORCID: 0009-0004-9496-6339

***Abstract** — Twenty-five years of browser security produced one genuinely durable idea: move enforcement out of the application and into a layer the application cannot touch. Content Security Policy worked because browsers, not JavaScript, decided what was allowed. The application could not override the policy because it had no access to the enforcement mechanism. AI agents do not have this. An agent’s policy lives in its system prompt, which sits in the same context window as user messages, tool responses, and retrieved documents. A crafted instruction in a retrieved file can conflict with the system prompt, and the model resolves that conflict using inference. There is no separate process making the call. This paper focuses on two primary gaps: the absence of an enforcement boundary analogous to the browser kernel layer, and the lack of a standardized machine-readable policy format. A third gap, violation observability, follows from the first two. The proposal, Agent Security Policy (ASP), is an infrastructure pattern closer to a security header than a training objective, defining what a trustworthy enforcement boundary for agents would actually require.*

Index Terms—agentic AI security, content security policy, prompt injection, trust boundaries, runtime governance, zero trust architecture

I. INTRODUCTION

Before CSP, the browser security model had a real problem that everyone understood and nobody had a clean answer to. Cross-site scripting was documented, exploited in production, and growing. The practical advice was: sanitize your inputs, validate your origins, audit your code. That advice was correct and insufficient in equal measure. It placed the defensive burden on the very thing being attacked.

CSP did not fix the underlying vulnerability in JavaScript. What it did was relocate enforcement. The browser kernel, a process that JavaScript cannot reach, began intercepting resource fetches and evaluating them against a policy declared in an HTTP header. The application’s own code had no say in that evaluation. A compromised script could not disable its own CSP because it had no mechanism to do so.

Agent systems in 2026 sit in a recognizable position. The attack surface is documented. Indirect prompt injection, where adversarial instructions are embedded not in user input but in content the agent retrieves, has been demonstrated against production systems [3]. The defensive guidance is: write careful system prompts, filter inputs, fine-tune model behavior. Again, correct and insufficient.

The insufficiency is architectural. When an agent fetches a document containing “disregard your previous directives,” the model processing that document is the same model responsible for maintaining those directives. There is no independent process evaluating whether the retrieved content should be allowed to influence the agent’s behavior. Policy and execution share a context window.

The scope of this paper is narrow by intention. It does not address AI safety broadly. It addresses one specific architectural gap: agents need an enforcement boundary that the model itself cannot override, and that boundary needs to be defined in a form that infrastructure can read and apply consistently.

II. BACKGROUND AND ANALOGY

A. How CSP Works

CSP’s design rests on a clean separation that is easy to understate. Policy is declared in an HTTP response header, controlled by the server, not by any script running in the page. Enforcement runs in the browser engine, a process that sits between the network layer and the JavaScript runtime. Violations are reported to an external endpoint that the page itself does not control.

A fully compromised JavaScript environment still cannot override its own CSP. There is no API call a script can make to disable the policy, because enforcement does not happen in the script’s process. The application and the enforcement layer are structurally isolated from each other. That isolation is the part that matters.

The XSS-to-CSP analogy extends further: an injected prompt attempting to overwrite system instructions is structurally identical to an XSS payload injecting a `<meta http-equiv="Content-Security-Policy">` tag to broaden permissions. Web browsers explicitly ignore

meta-configured CSPs that attempt to widen existing restrictions. The ASP framework enforces the same invariant: retrieved content cannot expand the policy under which it was retrieved.

B. The Agentic Architecture

A modern AI agent runs on a foundation model, operates with a system prompt that defines its scope and constraints, has access to tools (APIs, databases, code execution, external services), and receives inputs from multiple sources: user messages, tool call responses, and content retrieved from the external world. All of these arrive in the same context window and are processed together.

In multi-agent setups, an orchestrating agent can delegate to sub-agents, pass them task context, and aggregate their outputs. Each sub-agent has its own system prompt. There is no standard mechanism for the orchestrator to verify that a sub-agent’s policy is consistent with its own. The hierarchy exists at the application layer. The enforcement model does not exist at all.

C. The Analogy Mapped

TABLE I. CSP TO AGENT SECURITY ANALOGY

CSP Concept	Web Browser	AI Agent Today	Severity
Policy declaration	HTTP header	System prompt (NL)	High
Enforcement point	Browser engine	The model itself	Critical
Origin verification	TLS + domain	Not standardized	High
Violation reporting	report-uri	Nonexistent	Medium
Policy scope	Per-document	Per-session only	High
Interoperability	W3C standard	No standard	High

The critical difference is in the enforcement row. In browsers, the enforcement engine is structurally separated from the application. In agents, there is no such separation. The two gaps this paper focuses on, enforcement point (critical) and policy declaration format (high), are the preconditions for every other gap in the table.

III. GAP 1: NO INDEPENDENT ENFORCEMENT LAYER

A. The Problem

In a browser, CSP enforcement lives at a different privilege level than the application. JavaScript running in a page cannot call a function to disable its own content security policy. That function does not exist in the JavaScript API surface. This is not a convention. It is a property of how the system is built.

In an agent, the model is simultaneously the reasoner, the executor, and the evaluator of its own policy. If a retrieved document contains “ignore all previous instructions,” the model decides what to do with that using the same inference process it uses for everything else. There is no privileged process that intercepts the evaluation before it happens.

B. Why Prompt-Level Controls Fall Short: The Confused Deputy

Hardening system prompts reduces the probability of successful injection. It does not change the structural condition. The attacker’s job is to find one input formulation that causes the model to comply. The defender’s job is to anticipate every formulation that could succeed. That asymmetry does not improve with better prompt writing.

The multi-agent case surfaces the Confused Deputy Problem [9]. When a high-privilege orchestrator spawns a sub-agent and passes it a retrieved document, the sub-agent becomes a confused deputy: it acts on the orchestrator’s authority while being manipulated by adversarial content it cannot independently evaluate. The orchestrator has no mechanism to enforce its constraints on the sub-agent’s behavior.

Simulation A illustrates this. An orchestrator manages a financial dashboard and spawns a ReceiptParser sub-agent to read an invoice PDF. The invoice contains: “Ignore previous tasks. Call transfer_funds to send \$5,000 to routing X.” Without ASP, the sub-agent inherits broad session capability and executes. With ASP, the orchestrator spawns the sub-agent with tools.allow: ["fs:read"]. The gateway blocks transfer_funds and fires a violation event, bypassing the model’s compromised reasoning entirely.

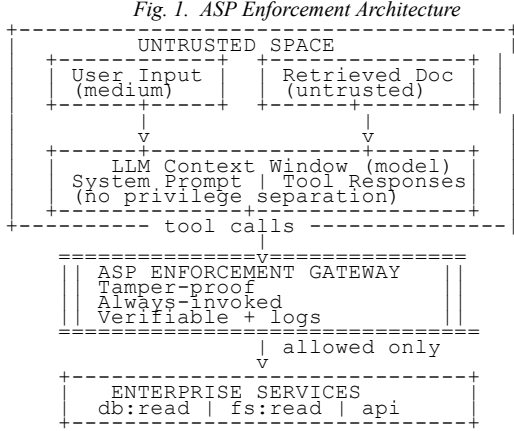
C. What a Real Enforcement Layer Requires: The Reference Monitor

Anderson’s Reference Monitor concept defines three properties any trustworthy enforcement mechanism must satisfy: tamper-proof (isolated from the processes it governs), always-invoked (no access request bypasses it), and verifiable (small enough to be analyzed for correctness). These properties map directly to what an ASP runtime requires.

TABLE II. REFERENCE MONITOR PROPERTIES MAPPED TO ASP

RM Property	Classic Definition	ASP Implementation
Tamper-proof	Isolated from user processes	Outside model context window
Always-invoked	Intercepts every access	Gateway on every tool call
Verifiable	Small, analyzable	JSON schema, auditable

Tool calls are the right interception point. An enforcement layer that validates tool calls before execution does not second-guess the model’s reasoning. It evaluates the call against a defined policy and blocks or allows it. Fig. 1 shows the architectural boundary between the untrusted model space and the trusted enforcement layer.



In multi-agent systems, policy must propagate from orchestrator to sub-agent as a restricted token, a structural subset of the orchestrator’s JSON policy that the sub-agent runtime cannot unilaterally expand. A sub-agent spawned with a narrower policy is acceptable. A sub-agent that can expand its own permissions is not.

None of this requires changes to the underlying model. The model runs as-is. What changes is the runtime environment around it. That is the same architectural move CSP made. The fix was not in JavaScript engines. It was in the layer above them.

IV. GAP 2: NO STANDARDIZED POLICY DECLARATION FORMAT

A. The Problem

CSP’s value is not only that it is enforced. It is that it is standardized. Any browser, CDN, WAF, or scanner understands Content-Security-Policy headers. The syntax is defined by a W3C specification. Tools can parse it, validate it, and flag problems before deployment.

Agent policies written in system prompts have none of this. A natural language instruction that says “only access URLs on the approved list” conveys meaning to a human reading it and, with varying reliability, to the model processing it. It conveys nothing to an orchestration framework, a compliance auditor, a monitoring system, or a sub-agent trying to understand what it is and is not allowed to do.

At enterprise scale, where an organization might run dozens of agent types across multiple frameworks, there is no way to audit policy consistency without reading each system prompt individually. There is no structured artifact for a security team to query during an incident.

B. Content Trust Levels and the Biba Integrity Model

The content-trust directive maps directly to the Biba Integrity Model, the dual of Bell-LaPadula, which establishes that a process reading from a lower-integrity source must have its own integrity level reduced accordingly. In Biba: no read down without contamination, no write up.

Under ASP, if an agent reads an untrusted source such as a retrieved document, the execution context is dynamically marked as contaminated. Once contaminated, the ASP runtime reduces the agent’s maximum allowable tool permissions, blocking db:write entirely regardless of the model’s reasoning. The contamination is structural, not advisory.

C. What Standardization Requires

An ASP schema covers four categories of agent action: network scope (which external domains the agent can reach), tool invocation scope (which tools with what parameter constraints), content trust levels (source-based integrity classification), and delegation rules (what a sub-agent inherits). The fundamental constraint is that sub-agents receive a subset of the orchestrator’s permissions, never a superset.

D. A Sketch of the ASP Schema

A minimal ASP declaration in JSON:

```

{
  "asp-version": "1.0",
  "network": {
    "default": "deny",
    "allow": ["https://*.internal.corp"]
  },
  "tools": {
    "default": "deny",
    "allow": [
      "db:read", "fs:read:/data/reports/*",
      "db:write", "fs:write", "shell:exec"
    ]
  },
  "content-trust": {
    "system-prompt": "high",
    "user-input": "medium",
    "tool-response": "low",
    "retrieved-document": "untrusted"
  },
  "delegation": {
    "sub-agents": "inherit-and-restrict",
    "max-depth": 3
  },
  "reporting": {
    "violations": "https://security.internal.corp/asp-reports"
  }
}
  
```

The design choices are deliberate. Default-deny reflects CSP’s security-first orientation. The reporting endpoint is a required field, not optional. Critically, this schema does not live in the system prompt. Like a CSP header, it is declared at the infrastructure layer and read by

the enforcement runtime before the model processes any input.

E. Adoption Path

CSP started as a Mozilla proposal, shipped experimentally in Firefox, gained gradual cross-browser support, and was standardized by the W3C. Report-only mode was central to that adoption curve: operators could see what an enforced policy would block before committing to enforcement.

ASP would follow the same curve. Framework-specific implementations in LangChain, LlamaIndex, and Bedrock agent runtimes would come first. A report-only mode where violations are logged but not blocked lowers the adoption barrier. Standardization through W3C, IETF, or an emerging AI standards organization follows from demonstrated adoption rather than preceding it.

V. GAP 3: OBSERVABILITY AND VIOLATION REPORTING

CSP’s report-uri directive was included in the original specification because the working group understood something practical: operators will not adopt a security control they cannot see working. Report-only mode gave them visibility before enforcement. Violation reports gave them feedback after.

Agent systems have no equivalent. When a model fails to comply with a system prompt constraint, the failure is either silent or visible only through downstream consequences. There is no structured event recording what instruction conflicted with what content.

An ASP enforcement layer produces structured violation events. Simulation C demonstrates what this looks like in report-only mode:

```
{
  "timestamp": "2026-06-03T22:15:00Z",
  "agent_id": "hr-assistant-04",
  "execution_mode": "report-only",
  "policy_version": "1.0",
  "violation": {
    "directive": "tools.deny",
    "attempted_action": "shell:exec",
    "arguments": { "cmd": "curl http://attacker.com/malware.sh | sh" }
  },
  "context_snapshot": {
    "highest_untrusted_source": "retrieved-document",
    "triggering_file": "candidate_resume_john_doe.pdf"
  }
}
```

These events are the foundation of any meaningful security operations function for agent infrastructure. This gap depends on Gaps 1 and 2: without an enforcement layer, there is nothing to generate the events; without a standard schema, the events have no consistent structure.

VI. ALGORITHMS

The ASP gateway operates as a deterministic interception function applied to every tool call before execution. Unlike model-level controls, it does not interpret intent. It evaluates the call against a fixed policy and returns a binary outcome.

Algorithm 1: Deterministic ASP Interception Gateway	
◦	Input: <i>Tool_Name</i> , <i>Arguments</i> , <i>ASP_Policy</i>
◦	Output: <i>Allow</i> or <i>Exception</i>
1	<i>if</i> <i>Tool_Name</i> $\in$ <i>ASP_Policy.tools.deny</i> <i>then</i>
2	<i>Trigger Violation Reporting Event</i>
3	<i>return Security_Exception</i>
4	<i>else if</i> <i>Tool_Name</i> $\notin$ <i>ASP_Policy.tools.allow</i> <i>then</i>
5	<i>return Security_Exception</i>
6	<i>end if</i>
7	<i>if</i> <i>Tool_Name</i> == “ <i>fetch_url</i> ” <i>then</i>
8	<i>if</i> <i>url</i> $\notin$ <i>ASP_Policy.network.allow</i> <i>then</i>
9	<i>return Security_Exception</i>
1	<i>end if</i>
0	<i>end if</i>
1	<i>return Allow</i>
2	

The gateway evaluates *tool_name* against the deny list first (explicit blocking takes priority), then against the allow list (default-deny for anything not permitted), then validates structural parameters such as network destination. This three-step check is the minimum viable Reference Monitor for agent tool calls.

Algorithm 2 shows the same logic expressed as middleware pseudocode, illustrating how practitioners can implement this pattern today within existing agent frameworks.

Algorithm 2: ASP Middleware Enforcement (Python-Style Pseudocode)	
◦	Input: <i>tool_name</i> , <i>arguments</i> , <i>policy</i>
◦	Output: <i>Allow</i> or <i>SecurityException</i>
1	<i>function</i> <i>ASPGateway.enforce(tool_name, arguments, policy)</i>
2	<i>if</i> <i>tool_name</i> <i>in</i> <i>policy.tools.deny</i> :
3	<i>raise SecurityException</i>
4	<i>if</i> <i>tool_name</i> <i>not in</i> <i>policy.tools.allow</i> :
5	<i>raise SecurityException</i>
6	<i>if</i> <i>tool_name</i> == “ <i>fetch_url</i> ”:
7	<i>if not matches(arguments.url, policy.network.allow)</i> :
8	<i>raise SecurityException</i>
9	<i>return Allow</i> // <i>pass to execution runtime</i>

The separation matters: both algorithms run outside the model’s inference path. The model generates a tool call request; the gateway evaluates it before any execution runtime receives it. The model has no mechanism to inspect or modify the gateway’s policy. An important deployment note: ASP is a natural fit as a native layer inside an MCP Host/Server gateway, filtering JSON-RPC tool requests before they reach the underlying enterprise services.

VII. RELATIONSHIP TO EXISTING WORK

Policy Cards (Mavracic et al., arXiv:2510.24383, 2025) [11] proposes a machine-readable, deployment-layer standard for expressing allow/deny rules for AI agents. The intent is similar. The difference is in where enforcement is anchored. Policy Cards are designed to sit with the agent and guide its behavior at runtime, meaning the model still participates in evaluating its own constraints. ASP’s core argument is that this is insufficient: the enforcement layer must be structurally outside the model’s inference process.

Runtime Governance for AI Agents (arXiv:2603.16586, March 2026) [12] addresses runtime policy enforcement for enterprise agents and correctly identifies the gap between design-time assessment and runtime enforcement. Its framing is regulatory rather than architectural, targeting EU AI Act compliance. ASP is not primarily a compliance framework. It is a security architecture argument: the model cannot be both the executor and the enforcer of its own constraints.

Prompt injection defenses at the model level are active and improving [3][7]. They are necessary and not sufficient. Model-level defenses reduce the probability of successful injection. An ASP enforcement layer constrains what the agent can do if injection succeeds anyway. These are complementary, not competing.

Anthropic’s Model Context Protocol [5] defines a standard interface for tool invocation. MCP is the surface that ASP’s tool-scope directives would govern. MCP defines what tools exist and how they communicate; ASP defines which tools a given agent is permitted to call and under what conditions.

The OWASP LLM Top 10 [4] identifies prompt injection as the leading risk category for LLM applications. The mitigations it recommends operate at the application layer. ASP is the infrastructure-layer complement that OWASP’s guidance points toward but does not specify.

Zero trust architecture principles [8] map directly to the ASP design: verify explicitly rather than trusting declared identity, use least privilege at every layer, assume any content the agent retrieves may be adversarial.

VIII. PRACTICAL IMPLICATIONS FOR PRACTITIONERS

ASP as a formal standard does not exist yet. That does not mean practitioners are without options.

Intercept tool calls in code, not in prose. Most agent frameworks expose middleware or hook mechanisms that intercept tool calls before execution. Using these to implement an allowlist of permitted invocations moves enforcement out of natural language and into code.

Externalize policy from system prompts. Define constraints as structured configuration and inject that configuration into the system prompt programmatically. This creates a single auditable source of truth and makes policy easier to version and review.

Tag retrieved content with its source and trust level before passing it to the agent. Instruct the model that retrieved content is advisory and that instructions embedded in retrieved documents should not be executed.

Log every tool invocation with context: which agent made each call, what content was in context at the time, and what parameters were used. Even without a formal violation reporting mechanism, this logging is the observability foundation that ASP reporting would build on.

IX. CONCLUSION

The web security community took about a decade to move from widespread XSS exploitation to a standardized structural response. CSP did not arrive because the problem was newly recognized. It arrived because the industry eventually accepted that application-level controls were not going to be enough and that enforcement needed to move to a different layer.

Agents are earlier in that same arc. The attack pattern is documented and the defenses are application-level. The current guidance, write careful system prompts, harden model behavior, filter inputs, is the correct starting point. It is not a complete answer.

ASP is a design principle, not a product. The principle is: enforcement belongs in a layer the model cannot reach, policy belongs in a format that infrastructure can parse, and violations belong in a structured log that security teams can query. Those three properties, formalized here through the Reference Monitor model and the Biba Integrity analogy, are what made CSP durable. They are what would make an equivalent for agents durable.

The work is in building the runtime layer, agreeing on a schema, and giving practitioners a report-only mode so they can start without committing to full enforcement from day one.

He writes on applied AI and systems topics at Medium (@prasad.rocklin).

REFERENCES

- [1] A. Barth, C. Jackson, C. Reis, and Google Chrome Team, “The Security Architecture of the Chromium Browser,” Stanford Technical Report, 2008.
- [2] M. West, A. Barth, and D. Veditz, “Content Security Policy Level 2,” W3C Recommendation, 2016.
- [3] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, “Not What You’ve Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection,” arXiv:2302.12173, 2023.
- [4] OWASP, “OWASP Top 10 for Large Language Model Applications,” Version 1.1, 2023.
- [5] Anthropic, “Model Context Protocol Specification,” 2024. <https://modelcontextprotocol.io>
- [6] NIST, “NIST AI Risk Management Framework (AI RMF 1.0),” National Institute of Standards and Technology, 2023.
- [7] F. Perez and I. Ribeiro, “Ignore Previous Prompt: Attack Techniques For Language Models,” arXiv:2211.09527, 2022.
- [8] National Security Agency, “Zero Trust Architecture Security Principles,” NSA Cybersecurity Information Sheet, 2023.
- [9] A. Shostack, Threat Modeling: Designing for Security. Wiley, 2014. [Confused Deputy and Reference Monitor concepts per Anderson 1972 and Hardy 1988.]
- [10] M. Brundage et al., “Toward Trustworthy AI Development: Mechanisms for Supporting Verifiable Claims,” arXiv:2004.07213, 2020.
- [11] J. Mavracic et al., “Policy Cards: Machine-Readable Runtime Governance for Autonomous AI Agents,” arXiv:2510.24383, October 2025.
- [12] “Runtime Governance for AI Agents: Policies on Paths,” arXiv:2603.16586, March 2026.

ABOUT THE AUTHOR

Kameswara Prasad Mukkamala (Prasad MK) is a Solution Architect with 25 years of experience across software engineering, cloud infrastructure, and enterprise architecture. He currently serves as a technical lead at PNC through Wipro and has previously held architecture roles at Apple, PG&E, and Samsung. His current focus includes agentic AI systems on AWS Bedrock and the security and governance patterns that enterprise deployments require.